

Microservicio: Reception

1. Propósito

“Reception” es el punto de entrada del sistema Campus Seguro, su única responsabilidad es recibir las alertas de pánico generadas por los tótems ESP32 (vía MQTT) o por clientes HTTP (para pruebas sin hardware), validarlas mínimamente y encolarlas en Redis para que el resto de los microservicios las procesen de forma asíncrona.

No clasifica prioridad, no resuelve geolocalización ni persiste nada en base de datos: su trabajo termina en el momento en que la alerta queda en la cola “queue:alerts”.

2. Responsabilidades

- Suscribirse al broker MQTT (Mosquitto) y escuchar el topic.
- Exponer un endpoint HTTP para simular alertas sin necesidad de un tótem físico.
- Validar que la alerta tenga los campos obligatorios (device_id, latitude, longitude, emergency_type).
- Normalizar el timestamp de la alerta: ignora el reloj del ESP32 y sustituye timestamp por la hora exacta del servidor.
- Encolar la alerta validada en Redis (LPUSH queue:alerts) para que “geolocation” la consuma.
- Exponer GET /health para monitoreo/orquestación.

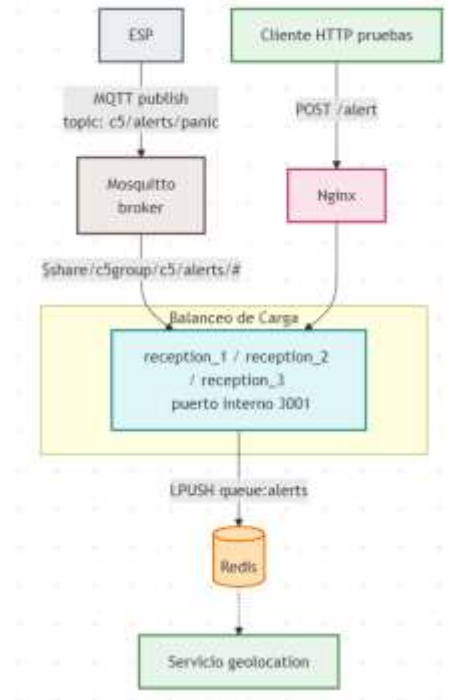
“Reception” corre replicado en 3 instancias (reception_1, reception_2, reception_3) detrás del balanceador Nginx.

Esto cumple dos objetivos:

1. **Alta disponibilidad:** Si un tótem en una zona del campus dispara muchas alertas a la vez, ninguna instancia se satura sola.
2. **Tolerancia a fallos del lado MQTT:** Las 3 instancias se suscriben al mismo topic usando un shared subscription de MQTT5, de modo que cada mensaje MQTT lo procesa una sola instancia, evitando alertas duplicadas en la cola.

Para tráfico HTTP, en cambio, es Nginx quien decide a cuál de las 3 instancias enviar cada request (least_conn).

4. Arquitectura e integración



“geolocation” enriquece la alerta con la zona del campus y llama a priority vía gRPC.

5. Variables de entorno

Variable	Default	Descripción
PORT	3001	Puerto donde escucha el servidor HTTP Express
INSTANCE_ID	reception_unknown	Identificador de la instancia (usado en logs y en el campo <code>processed_by</code> de cada alerta). En <code>docker-compose.yml</code> se fija a <code>reception_1</code> , <code>reception_2</code> , <code>reception_3</code>
MQTT_BROKER	mqtt://localhost:1883	URL del broker Mosquitto
REDIS_URL	redis://localhost:6379	URL de conexión a Redis

6. API HTTP

GET /health

Healthcheck simple, útil para monitoreo y para depends_on en Docker.

Respuesta 200 OK

```
{  
  "status": "ok",  
  "instance": "reception_2",  
  "timestamp": "2026-06-16T18:42:01.123Z"  
}
```

POST /alert

Permite simular una alerta sin necesidad de un tótem ESP32 físico, pensado para pruebas y demos.

Body:

Campo	Tipo	Obligatorio	Descripción
device_id	string	Sí	ID único del tótem (ej. PASILLO)
latitude	number	Sí	Latitud del tótem
longitude	number	Sí	Longitud del tótem
emergency_type	string	Sí	Tipo de emergencia (debe coincidir con alguno de los tipos definidos ej. robo_con_violencia, incendio, acoso_violencia)
location_name	string	No	Nombre descriptivo del lugar, usado por

			geolocation para mapear la zona
campus	boolean	No	Flag informativo de que la alerta viene del campus

Ejemplo de request:

```
curl -X POST http://localhost:8080/alert \
-H "Content-Type: application/json" \
-d '{
  "device_id": "PASILLO",
  "location_name": "Edificio Laboratorios -Quimica",
  "latitude": 19.9128,
  "longitude": -99.5790,
  "emergency_type": "robo_con_violencia",
  "campus": true
}'
```

Respuesta 200 OK:

```
{
  "message": "Alerta recibida",
  "instance": "reception_3",
  "alert": { "device_id":
    "TOTEM-LAB-A1",
    "location_name": "Edificio Laboratorios - Ala A",
    "latitude": 19.9128,
    "longitude": -99.5790,
    "emergency_type": "robo_con_violencia",
    "campus": true,
```

```
    "timestamp": "2026-06-16T18:42:01.123Z",  
    "received_at": 1781000521123,  
    "processed_by": "reception_3",  
    "source": "http"  
  }  
}
```

Respuesta 400 Bad Request (por falta de un campo obligatorio):

```
{ "error": "Campo requerido: emergency_type" }
```

A diferencia del flujo MQTT, en este endpoint Reception **no descarta silenciosamente** una alerta inválida, sino que responde 400 explícitamente, es una asimetría intencional para favorecer pruebas/depuración vía HTTP frente a la naturaleza "fire-and-forget" de MQTT.

7. Flujo MQTT

Suscripción

Al arrancar, cada instancia se conecta a Mosquitto y se suscribe usando shared subscription:

```
$share/c5group/c5/alerts/#
```

Esto significa que las 3 instancias comparten el grupo c5group: Mosquitto reparte cada mensaje publicado en c5/alerts/# a **solo una** de las instancias suscritas, no a las tres.

Es el mecanismo que reemplaza a un load balancer para el canal MQTT.

Con un payload JSON como:

```
{  
  "device_id": "TOTEM-LAB-A1",  
  "location_name": "Edificio Laboratorios - Ala A",  
  "latitude": 19.9128,  
  "longitude": -99.5790,  
  "emergency_type": "robo_con_violencia",  
}
```

```
"location_method": "totem_fijo",  
"campus": true  
}
```

Procesamiento del mensaje

Al llegar un mensaje al topic suscrito, Reception:

1. Parsea el JSON del payload.
2. Valida los 4 campos obligatorios (device_id, latitude, longitude, emergency_type). Si falta alguno, **descarta el mensaje silenciosamente**
3. Si es válido, construye el objeto alert añadiendo:
 - timestamp: hora del servidor en formato ISO 8601.
 - received_at: epoch en milisegundos de cuándo Reception recibió el mensaje (Date.now()), útil para medir latencia end-to-end en los servicios siguientes.
 - processed_by: el INSTANCE_ID de la instancia que lo procesó.
 - topic: el topic MQTT original.
4. Hace LPUSH queue:alerts en Redis con el JSON serializado del objeto alert.

8. Contrato de salida (queue:alerts)

Cada elemento que Reception inserta en la lista de Redis queue:alerts es un JSON con esta forma mínima garantizada:

```
{  
  "device_id": "string (obligatorio)",  
  "latitude": "number (obligatorio)",  
  "longitude": "number (obligatorio)",  
  "emergency_type": "string (obligatorio)",  
  "timestamp": "ISO 8601 — generado por Reception",  
  "received_at": "epoch ms — generado por Reception",  
  "processed_by": "instance_id de Reception",  
  "location_name": "string (opcional, viene del payload original)",  
  "campus": "boolean (opcional, viene del payload original)",  
  "topic": "string (solo presente si vino por MQTT)",  
}
```

"source": "'http' (solo presente si vino por POST /alert)"

}

9. Manejo de errores y casos límite

Situación	Comportamiento actual
Falta un campo obligatorio (MQTT)	Se descarta el mensaje, se loguea console.warn. No hay notificación ni reintento.
Falta un campo obligatorio (HTTP)	Responde 400 con el nombre del campo faltante.
JSON malformado en el mensaje MQTT	try/catch captura el error de JSON.parse, se loguea y se descarta.
Redis no disponible	ioredis reintenta la conexión automáticamente, mientras tanto, los LPUSH fallarán y se perderá la alerta en curso. No hay buffer local de respaldo.
Mosquitto no disponible al arrancar	El cliente MQTT reintenta conexión automáticamente. El servicio HTTP sigue funcionando con normalidad mientras tanto.
Doble entrega del mismo mensaje MQTT	Mitigado por shared subscription (cada mensaje va a una sola instancia), pero MQTT en QoS por defecto no garantiza exactly-once a nivel de broker.

10. Despliegue

Dockerfile

Build simple:

instala dependencias de producción (npm install --production) y copia el código fuente. No hay multi-stage build ni usuario no-root configurado.

docker-compose

```
reception_1:
  build: ./services/reception
  container_name: c5_reception_1
  environment:
    - PORT=3001
    - INSTANCE_ID=reception_1
    - MQTT_BROKER=mqtt://mosquitto:1883
    - REDIS_URL=redis://redis:6379
  depends_on:
    - mosquitto
    - redis
```

Ninguna instancia expone puerto directamente al host: solo es alcanzable a través de Nginx en el puerto 8080, o internamente vía MQTT.

Balanceo (Nginx)

```
http {
    upstream reception_cluster {
        least_conn;
        server reception_1:3001;
        server reception_2:3001;
        server reception_3:3001;
    }
}
```

least_conn envía cada request HTTP entrante a la instancia con menos conexiones activas en ese momento relevante sobre todo para el endpoint /alert usado en pruebas, y como vía de acceso al /health de cada instancia.

11. Dependencias (package.json)

Paquete	Versión	Uso
express	4.18.2	Servidor HTTP / API REST
mqtt	5.3.4	Cliente MQTT
ioredis	5.3.2	Cliente Redis para LPUSH en queue:alerts

12. Cómo probarlo localmente

Levantar solo lo necesario para Reception:

```
docker compose up --build mosquitto redis reception_1 reception_2 reception_3  
nginx
```

Healthcheck:

```
curl http://localhost:8080/health
```

Enviar alerta de prueba por HTTP:

```
curl -X POST http://localhost:8080/alert \  
-H "Content-Type: application/json" \  
-d '{"device_id":"TOTEM-LAB-A1","latitude":19.9128,"longitude":-  
99.5790,"emergency_type":"robo_con_violencia"}'
```

Verificar que llegó a la cola de Redis:

```
docker exec -it c5_redis redis-cli LRange queue:alerts 0 -1
```

13. Limitaciones conocidas

- No hay autenticación en POST /alert ni en la conexión MQTT (cualquiera en la red puede publicar alertas falsas).
- Los mensajes MQTT inválidos se descartan sin dejar rastro persistente (no hay dead-letter queue), lo que dificulta auditar tótems con firmware roto.
- No hay rate limiting: un tótem en bucle de fallos podría inundar queue:alerts.
- El INSTANCE_ID se usa solo para logging y trazabilidad, no hay un endpoint de métricas que reporte alertas procesadas por instancia.